

Add images to your Android app

1. Introduction
2. Set up your app
3. Add an `ImageView`
4. Adopt good coding practices
5. Solution code
6. Summary
7. Learn more
8. Practice on your own

About this codelab

subject Last updated Sep 22, 2023

account_circle Written by Google Developers Training team

1. Introduction

In this codelab you will learn how to add images to your app using an `ImageView`.

Prerequisites

- How to create and run a new app in Android Studio.
- How to add and remove, and set attributes on `TextViews` using the Layout Editor.

What you'll learn

- How to add an image or photo to your Android app.
- How to display an image in your app using an `ImageView`.
- How to extract text into a string resource to make it easier to translate your app and to reuse strings.
- How to make your app usable for the largest number of people.

What you'll build

- Extend the Happy Birthday app to add an image.

What you need

- A computer with Android Studio installed.
- The app from the Create a Birthday Card codelab.

2. Set up your app

1. Open your project from the previous codelab in Android Studio. You can use the [solution code](#) or the code you created. When you run the app, it should look like this.

2:03



Happy Birthday

Happy Birthday, Sam!

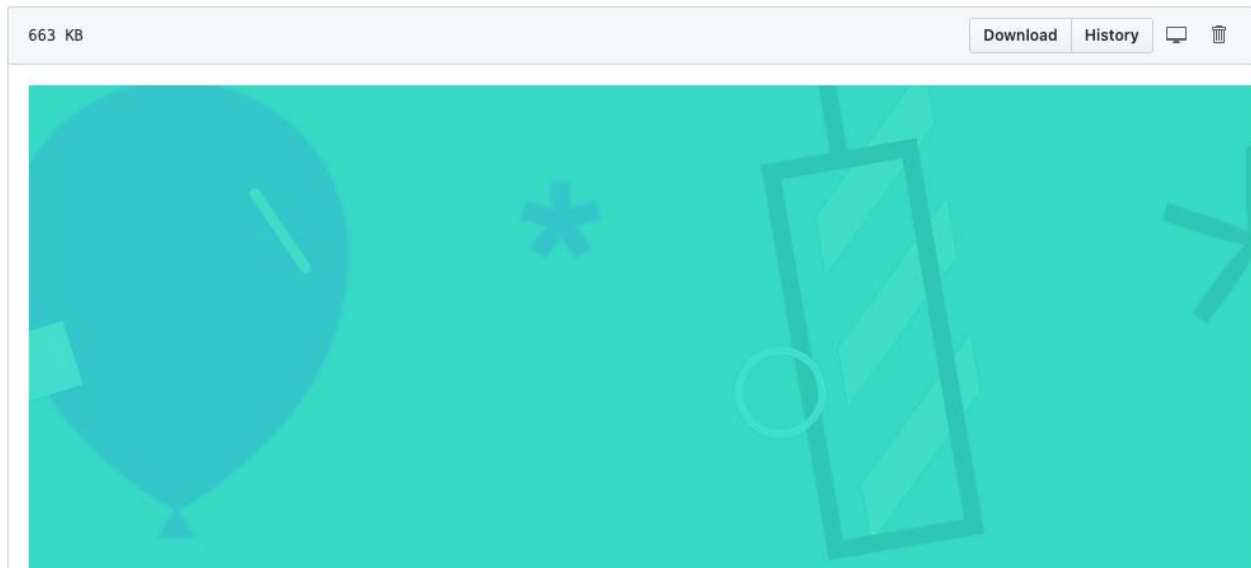
From Emma.



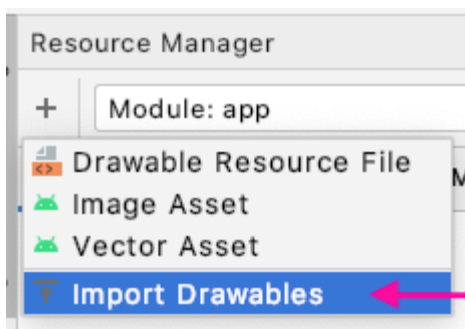
Add an image to your project

In this task, you'll download an image from the internet and add it to your Happy Birthday app.

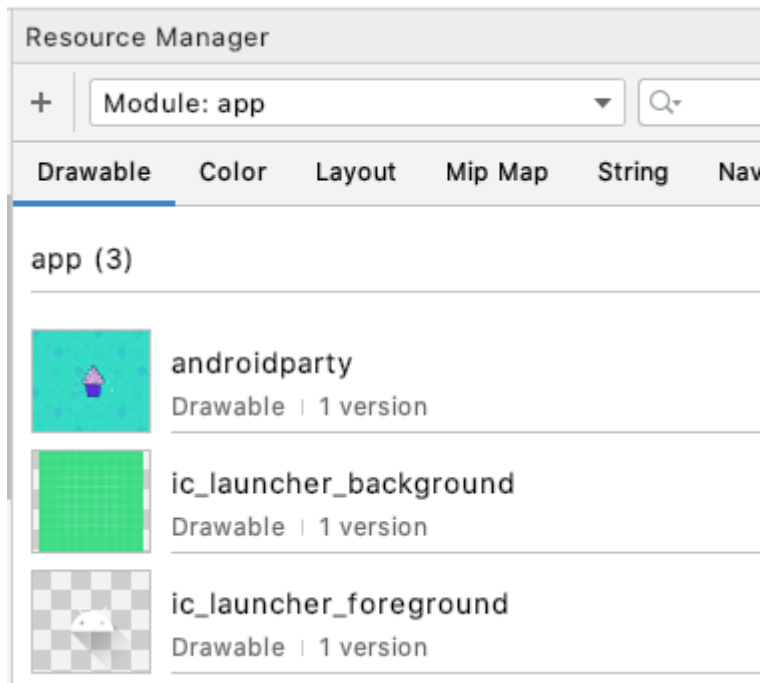
1. Click [here](#) to access the image for your birthday card on Github.
2. Click the **Download** button on the right. This displays the image by itself.



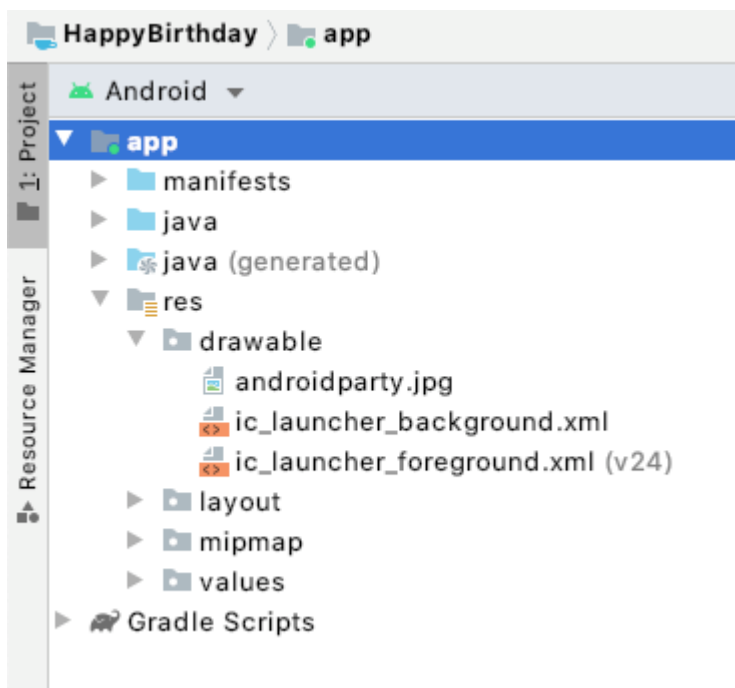
3. Right-click on the image and save the file to your computer as **androidparty.png**. Make note of where you saved it (for example, the **Downloads** folder).
4. In Android Studio, click on **View > Tool Windows > Resource Manager** in the menus or click on the **Resource Manager** tab to the left of the **Project** window.
5. Click the + below **Resource Manager**, and select **Import Drawables**. This opens a file browser.



6. In the file browser find the image file you downloaded and click **Open**.
7. Click **Next**. Android Studio shows you a preview of the image.
8. Click **Import**.
9. If the image has been imported successfully, Android Studio adds the image to your **Drawable** list. This list includes all your images and icons for the app. You can now use this image in your app.



10. Switch back to the project view by clicking on **View > Tool Windows > Project** in the menus or the **Project** tab on the far left.
11. Confirm the image is in the **drawable** folder of your app by expanding **app > res > drawable**.



3. Add an ImageView

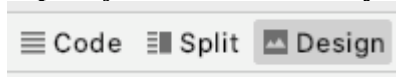
In order to display an image in your app, it needs a place to be displayed. Just like you use a `TextView` to display text, you can use an `ImageView` to display images.

In this task, you'll add an `ImageView` to your app, and set its image to the cupcake image you downloaded. Then you'll position it and adjust its size so it fills the screen.

Add an ImageView and set its image

1. In the **Project** window open `activity_main.xml` (**app** > **res** > **layout** > `activity_main.xml`).

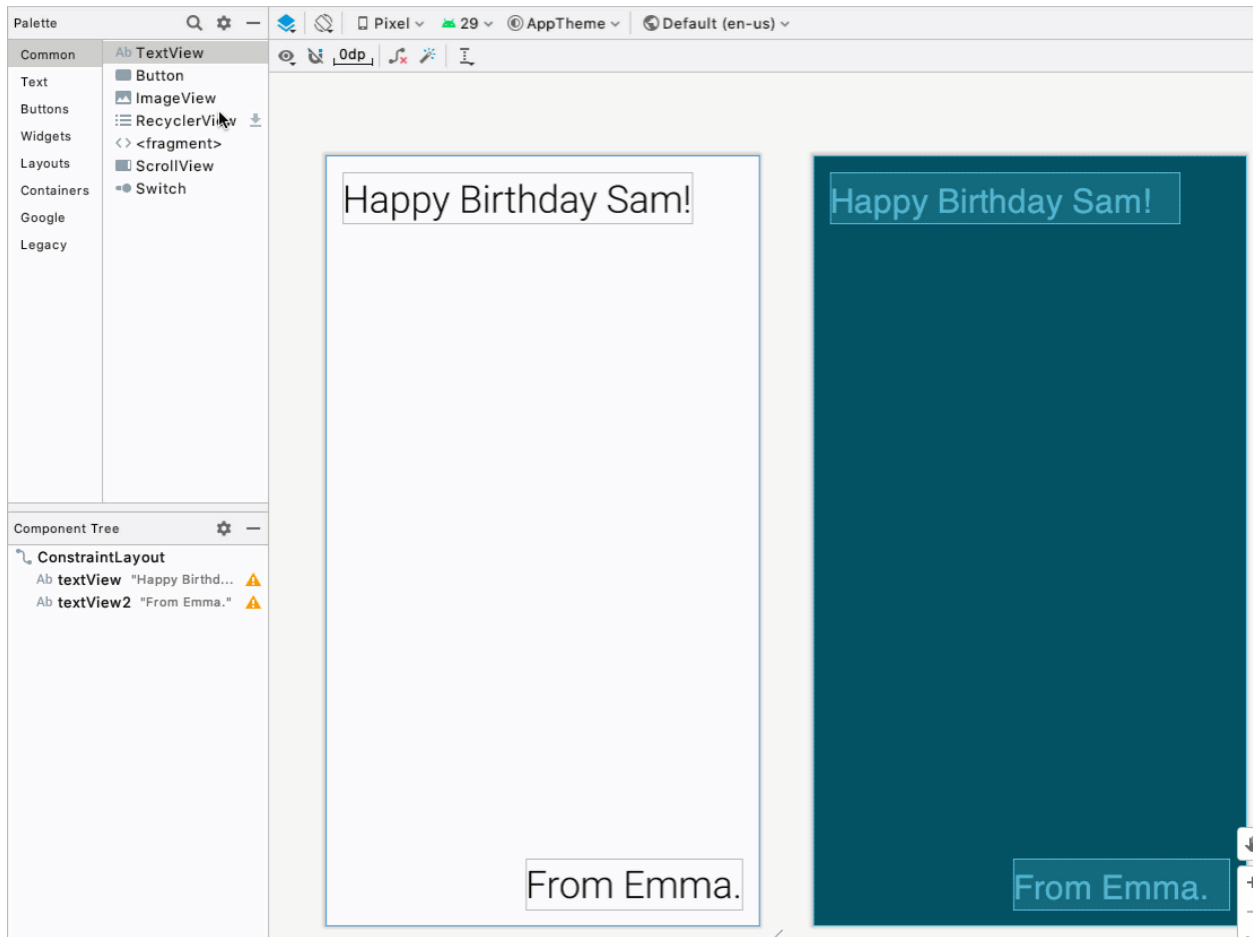
Tip: If you don't see the **Layout Editor**, click on the **Design** mode button in the upper right.



2. In the **Layout Editor**, go to the **Palette** and drag an `ImageView` to your app. Drop it near the center and not overlapping any of the text

The **Pick a Resource** dialog opens. This dialog lists all the image resources available to your app. Notice the birthday image listed under the **Drawable** tab. A *drawable resource* is a general concept for a graphic that can be drawn to the screen. It includes images, bitmaps, and icons and many other types of drawn resources.

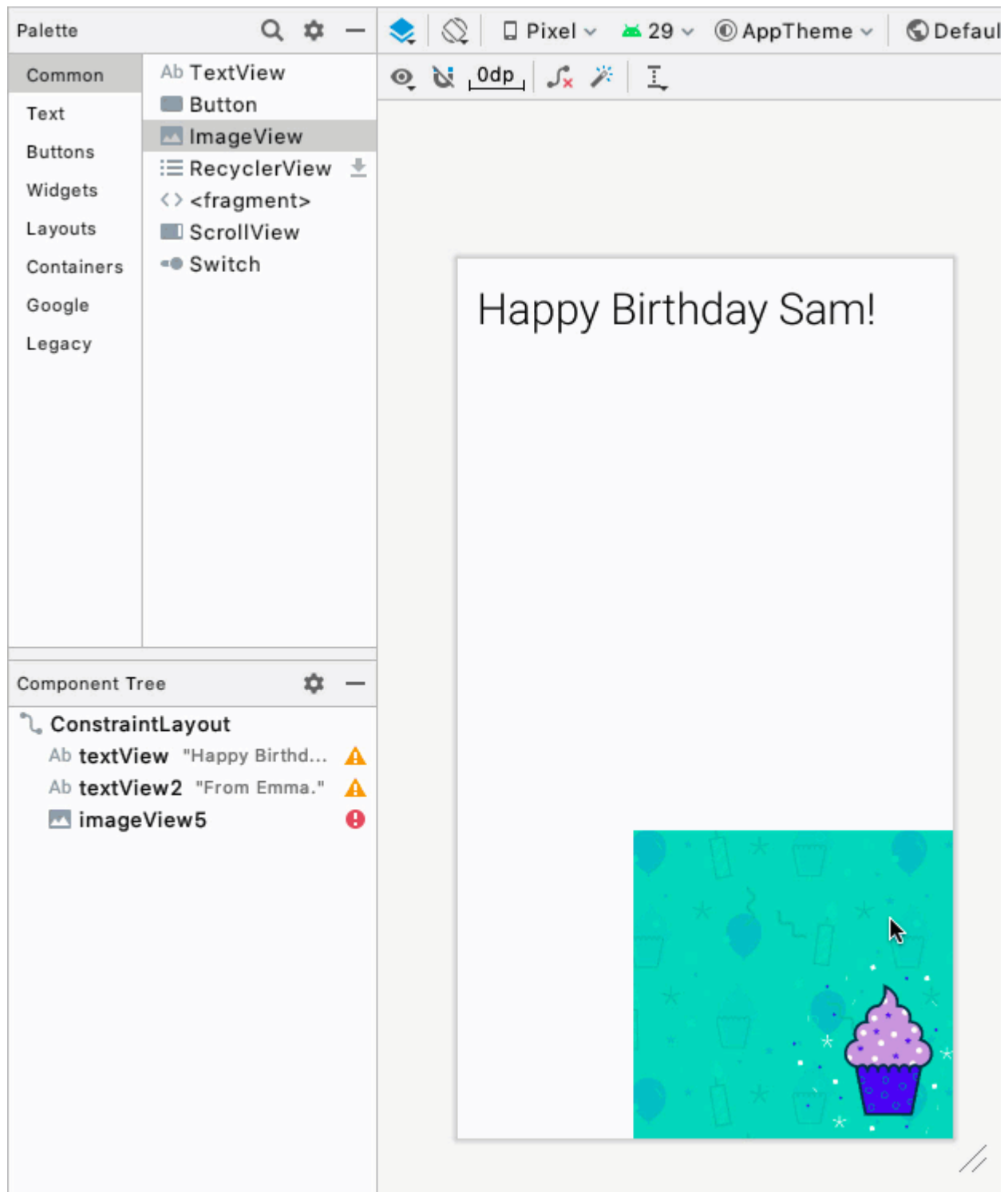
3. In the **Pick a Resource** dialog find the cake image in the **Drawable** list.
4. Click on the image and then click **OK**.



This adds the image to your app, but it's probably not in the right place and it doesn't fill the screen. You'll fix that in the next step.

Position and size the ImageView

1. Click and drag the `ImageView` around in the **Layout Editor**, and note that as you drag, a pink rectangle appears around the app screen in the **Design** view. The pink rectangle indicates the boundaries of the screen for positioning your `ImageView`.
2. Drop the `ImageView` so that the left and right edges align with the pink rectangle. Android Studio will "snap" the image to the edges when you're close. (You'll take care of the top and bottom in a moment.)

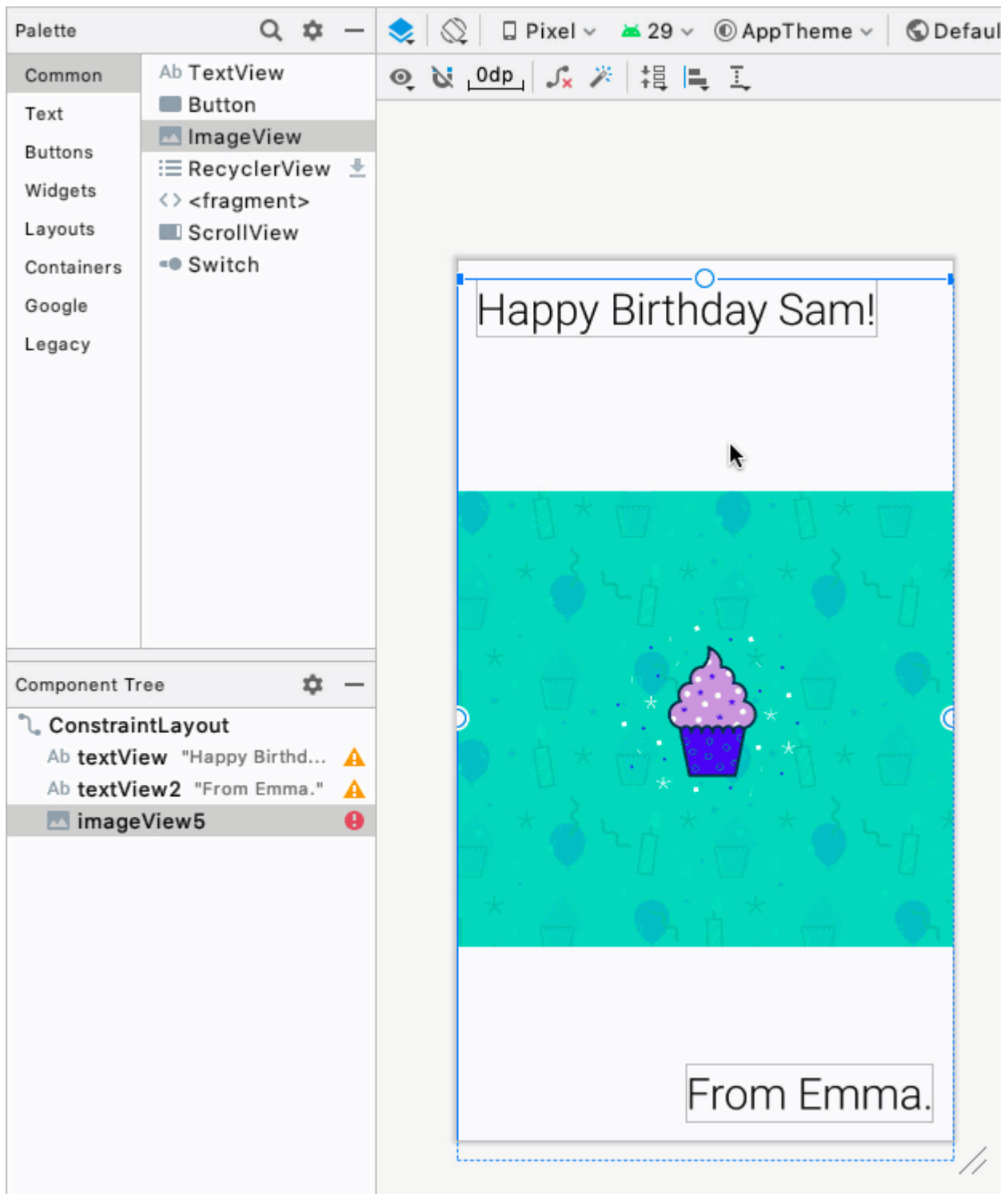


Views in a `ConstraintLayout` need to have horizontal and vertical constraints to tell the `ConstraintLayout` how to position them. You'll add those next.

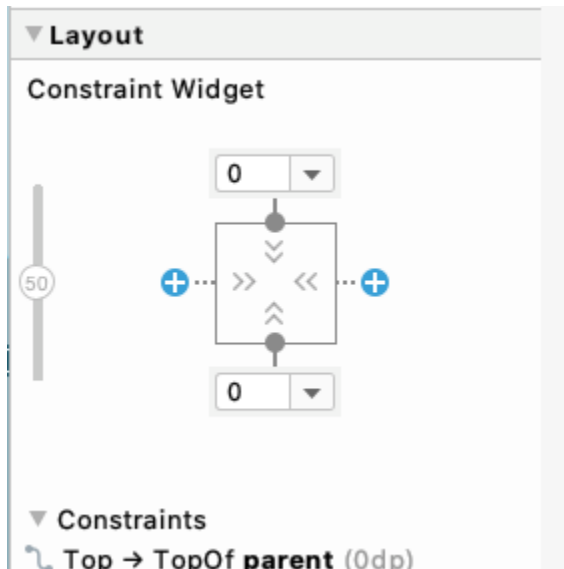
Note: A constraint in the context of the **Layout Editor** represents a connection or alignment of a view to another view, the parent layout, or an invisible guideline. For example, a view can be

constrained to be a certain distance from the edge of its container, or always be to the right of another view, or always the top view inside a container.

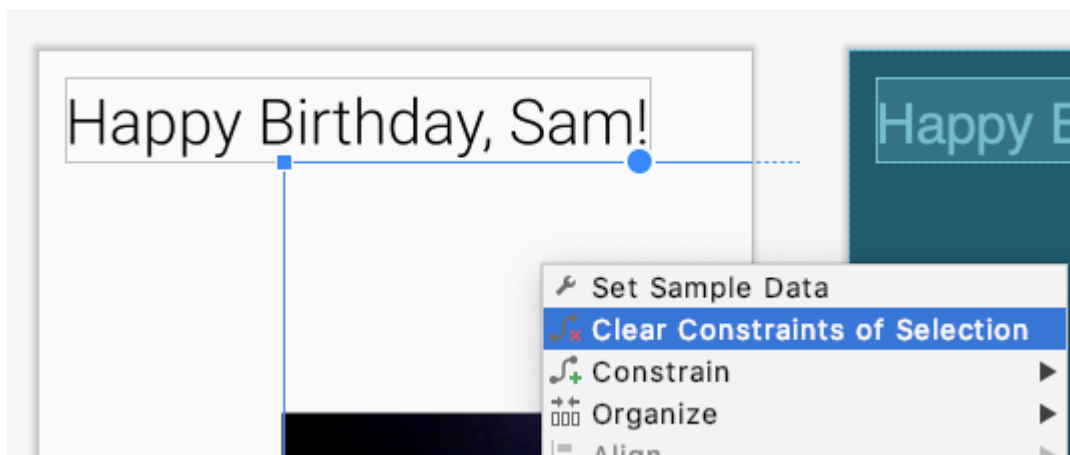
3. Hold the pointer over the circle at the top of the outline of the `ImageView`, and it highlights with another circle.
4. Drag the circle towards the top of the app screen, and an arrow connects the circle to your pointer as you are dragging. Drag until it snaps to the top of the screen. You've added a constraint from the top of the `ImageView` to the top of the `ConstraintLayout`.



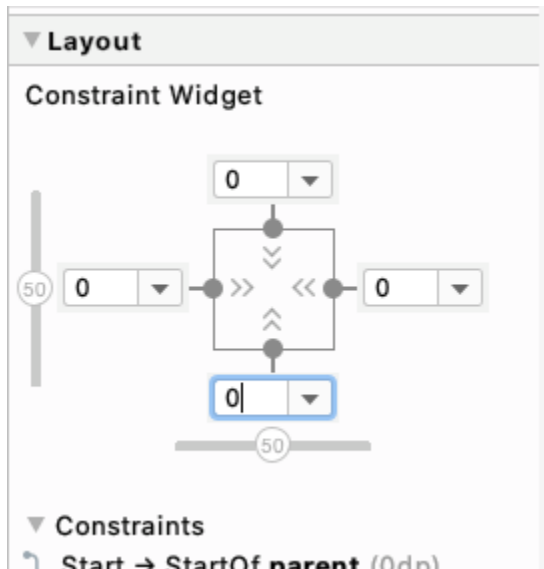
5. Add a constraint from the bottom of the `ImageView` to the bottom of the `ConstraintLayout`. It may be too close to the edge to drag as you did for the top. In that case, you can click on the bottom **+** in the **Constraint Widget** in the **Attributes** window to add a constraint. Make sure the margin is 0.



Tip: Adding constraints can be more difficult as you add more Views on the screen. If you add a constraint to the wrong View, you can undo that step with **Control+Z** (**Command+Z** on a Mac.) Or right-click on the View which will pop up a menu, and choose **Clear Constraints of Selection** which will remove all the constraints on that View.



6. In the **Attributes** pane, use the **Constraint Widget** to add a margin of 0 to the left and to the right sides. This automatically creates a constraint in that direction.



The image is now centered, but it isn't taking up the whole screen yet. You'll fix that in the next steps:

- Below the **Constraint Widget** in the **Constraints** section, set the **layout_width** to **0dp (match constraint)**. 0dp is a shorthand to tell Android Studio to use *match constraint* for the width of the `ImageView`. Because of constraints you just added, this makes it as wide as the `ConstraintLayout`, minus any margins.



- Set the **layout_height** to **0dp (match constraint)**. Because of the constraints you added, this makes the `ImageView` as tall as the `ConstraintLayout`, minus any margins.

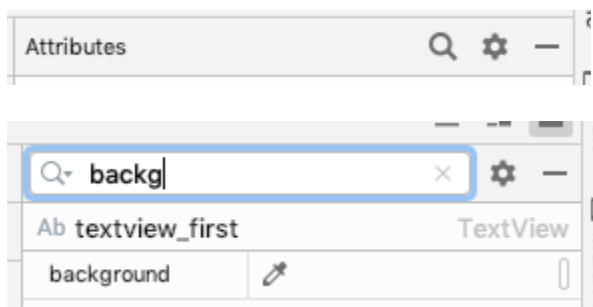


- The `ImageView` is as wide and tall as the app screen, but the image is centered inside the `ImageView` and there is a bunch of whitespace above and below the image. Since that doesn't look very attractive, you will adjust the **scaleType** of the `ImageView`, which says how to size and align the image. Read more about `ScaleType` in [the developer reference guide](#). Your app should now look as shown below.

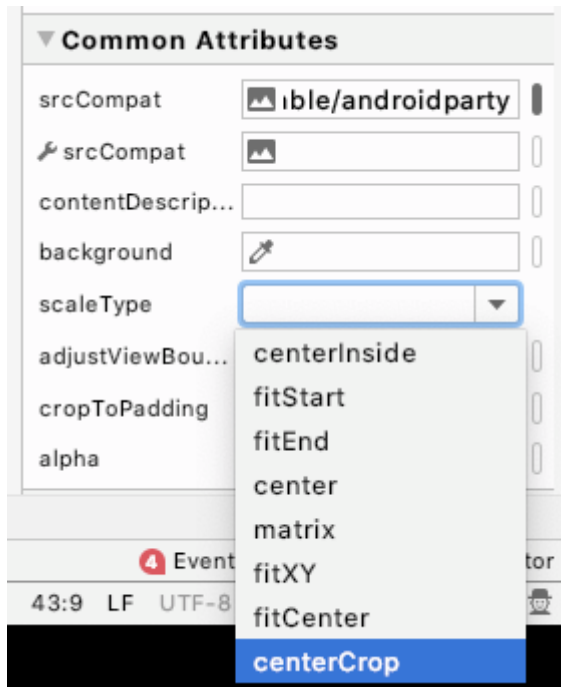


10. Find the **scaleType** attribute. You may need to scroll down or search for this attribute.
Try setting different values for the **scaleType** to see what they do.

Tip: To find a property in the list of all the properties, click on the magnifying glass icon to the right of Attributes, and begin typing the name of the property. Android Studio will show just the properties that contain that string.



11. When you're done, set the **scaleType** to **centerCrop** because that makes the image fill the screen without distorting it.



The cake image should now fill the entire screen, as shown below.

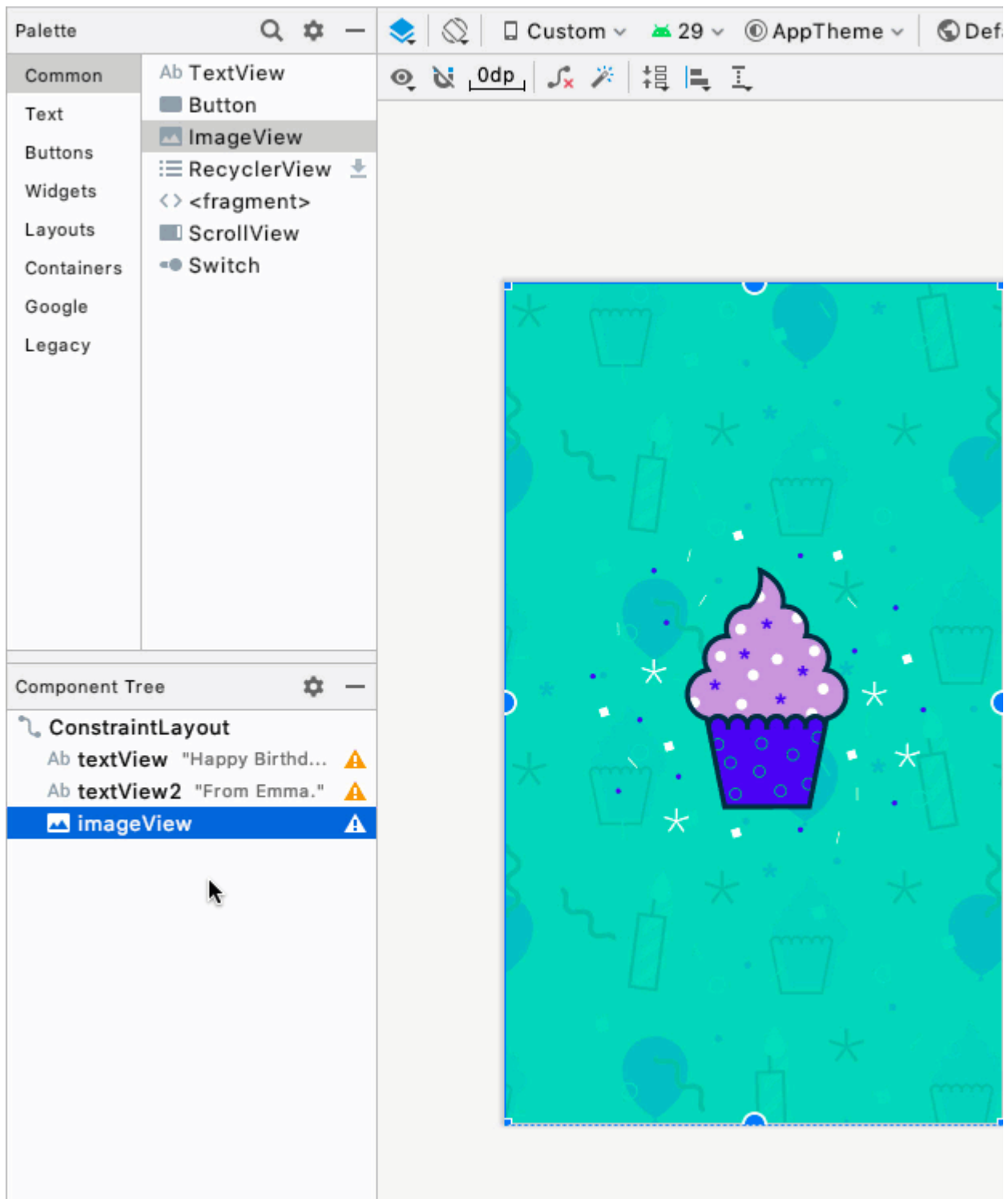


But now you can't see your birthday greeting and signature. You'll fix that in the next step.

Move the `ImageView` behind the text

After making the `ImageView` fill the screen, you can't see the text anymore. This is because the image now covers up the text. It turns out that the order of your UI elements matters. You added the `TextViews` first, then you added the `ImageView`, which means it went on top. When you add views to a layout, they are added at the end of a list of views, and they get drawn in the order they are in the list. The `ImageView` was added to the end of the list of `Views` in the `ConstraintLayout`, which means it's drawn last, and draws over the `TextViews`. To fix that, you'll change the order of the views and move the `ImageView` to the beginning of the list so it's drawn first.

In the **Component Tree**, click on the `ImageView`, and drag it above the `TextViews` to just below the `ConstraintLayout`. A blue line with a triangle appears showing where the `ImageView` will go. Drop the `ImageView` just below the `ConstraintLayout`.



The `ImageView` should now be first in the list below the `ConstraintLayout`, with the `TextView`s after it. Both the "Happy Birthday, Sam!" and the "From Emma." text should now be visible. (Ignore the warning for the missing content description for now.)

Congratulations! You've added an image to your Android app!

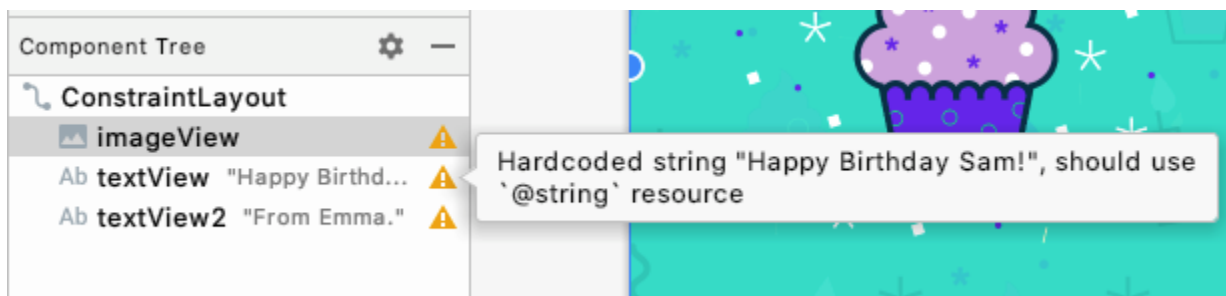
4. Adopt good coding practices

When you added the `TextView`s in the previous codelab, Android Studio flagged them with warning triangles. In this step, you'll fix those warnings and also fix the warning on the `ImageView`.

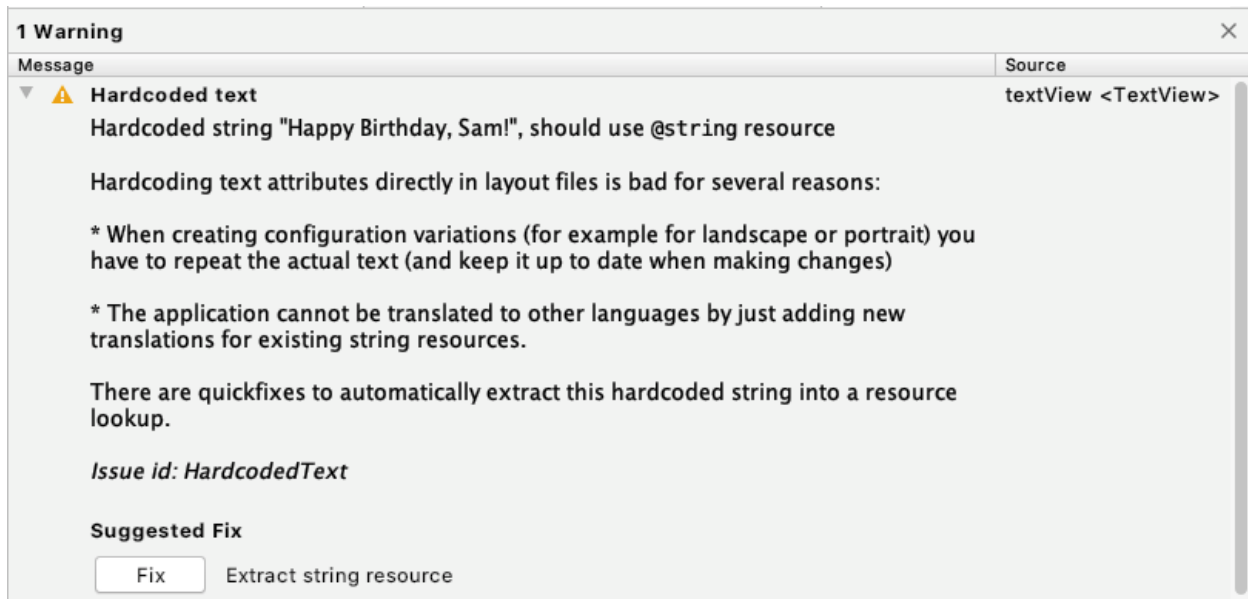
Translating

When you write apps, it's important to remember that they may be translated into another language at some point. As you learned in an earlier codelab, a string is a sequence of characters like "Happy Birthday, Sam!".

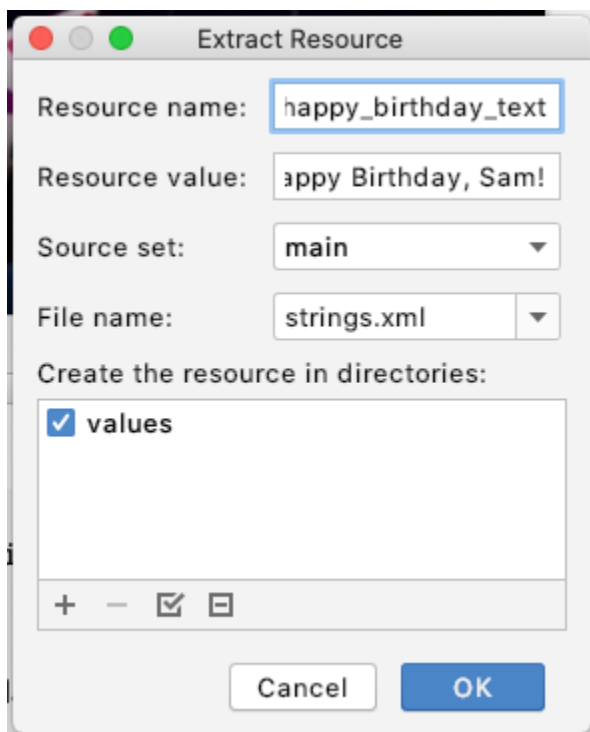
A *hardcoded* string is one that is written directly in the code of your app. Hardcoded strings make it more difficult to translate your app into other languages, and harder to reuse a string in different places in your app. You can deal with those issues by "extracting strings into a resource file". That means instead of hardcoding the string in your code, you put the string into a file, give it a name, and then use the name whenever you want to use this string. The name will stay the same, even if you change the string or translate it to a different language.



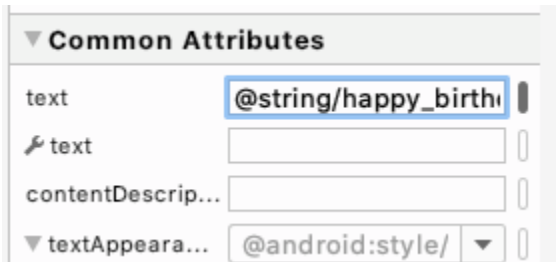
1. Click on the orange triangle next to the first `TextView` with "Happy Birthday, Sam!" Android Studio opens a window with more information and a suggested fix. You may need to scroll down to see the **Suggested Fix**.



2. Click the **Fix** button. Android Studio opens the **Extract Resource** dialog. In this dialog, you can customize what your string resource is called, and some details on how to store it. The **Resource name** is what the string is going to be called. The **Resource value** will be the actual string itself.
3. In the **Extract Resource** dialog, change the **Resource name** to **happy_birthday_text**. String resources should have lowercase names, and multiple words should be separate with an underscore. Leave the other settings with the defaults.



4. Click the **OK** button.
5. In the **Attributes** pane, find the **text** attribute, and notice that Android Studio has automatically set it to **@string/happy_birthday_text** for you.



6. Open **strings.xml** (**app > res > values > strings.xml**) and notice that Android Studio has created a string resource called **happy_birthday_text**.

```
<resources>
    <string name="app_name">Happy Birthday</string>
    <string name="happy_birthday_text">Happy Birthday, Sam!</string>
</resources>
```

The `strings.xml` file has a list of strings that the user will see in your app. Note that the name of your app is also a string resource. By putting the strings all in one place, you can more easily translate all the text in your app, and more easily reuse a string in different parts of your app.

7. Follow the same steps to extract the text for the signature `TextView`, and name the string resource **signature_text**.

Your finished file should look like this.

```
<resources>
    <string name="app_name">Happy Birthday</string>
    <string name="happy_birthday_text">Happy Birthday, Sam!</string>
    <string name="signature_text">From Emma.</string>
</resources>
```

Check your app for accessibility

Following good coding practices for accessibility allows all of your users, including users with disabilities, to navigate and interact with your app more easily.

Note: Android provides many tools for users. For example, [TalkBack](#) is the Google screen reader included on Android devices. TalkBack gives users spoken feedback so that users can use their device without looking at the screen. Read more about accessibility in [the developer reference guide](#).

Android Studio provides hints and warnings to help you make your app more accessible.

1. In the **Component Tree**, notice the orange triangle next to the `ImageView` that you added earlier. If you hover the pointer over it, you'll see a tooltip with a warning about a missing 'contentDescription' attribute on the image. A content description can help make your app more usable with TalkBack by defining the purpose of the UI element.

However, the image in this app is only included for decorative purposes. Instead of setting the content description that is announced to the user, you can just tell TalkBack to skip the `ImageView` by setting its **importantForAccessibility** attribute to **no**.

2. In the **Component Tree**, select the `ImageView`.
3. In the **Attributes** window, in the **All Attributes** section, find **importantForAccessibility** and set it to **no**.

The orange triangle next to the `ImageView` disappears.

4. Run your app again to make sure it still works.

Congratulations! You've added an image to your Happy Birthday app, followed the accessibility guidelines, and made it easier to translate to other languages!

2:16



Happy Birthday

Happy Birthday Sam!



From Emma.

5. Solution code

The solution code for the Happy Birthday app has been uploaded to GitHub, in case you'd like to see the code that we ended up with.

GitHub is a service that allows developers to manage code for their software projects. It uses Git, which is a version control system that keeps track of the changes made for each version of the code. If you've ever seen the version history of a Google Doc, you can see when and what edits were made on the document in the past. Similarly, you can track the version history of the code in a project. This is very helpful if you are working on a project as an individual or with a team.

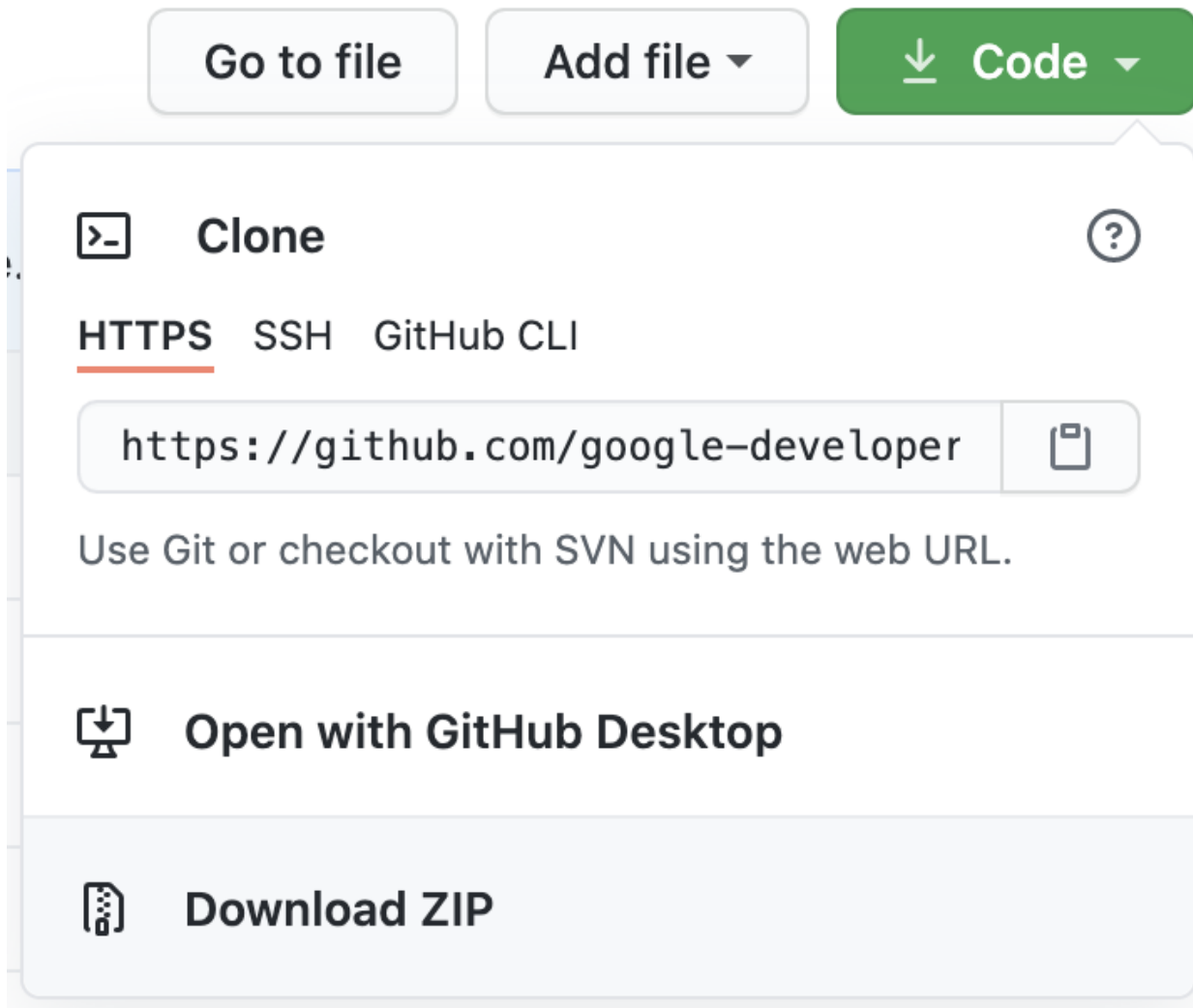
GitHub also has a website where you can view and manage your project. This GitHub link allows you to browse the Happy Birthday project files online or download them to your computer.

Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-birthday-card-with-image-app-solution>

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

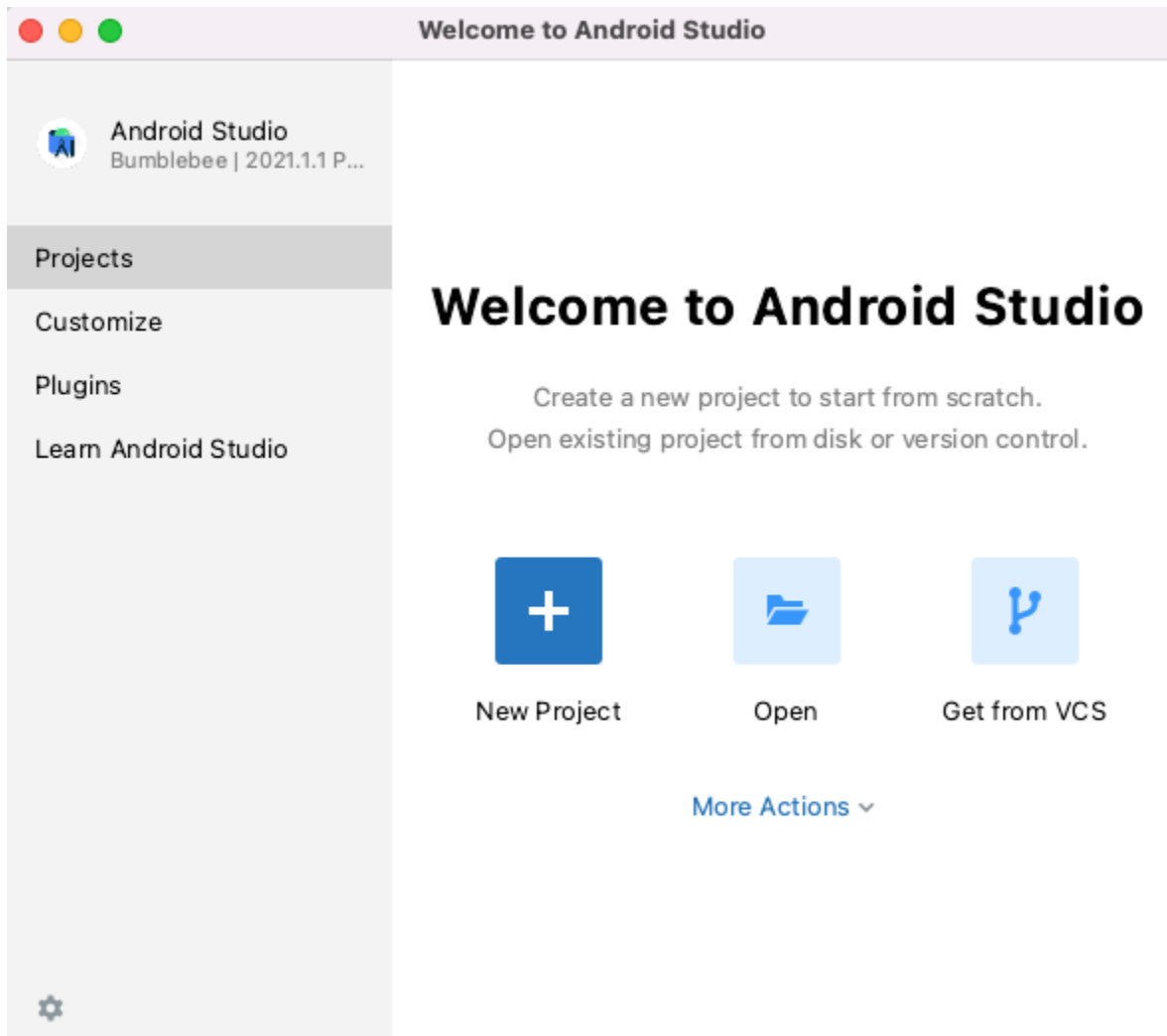
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



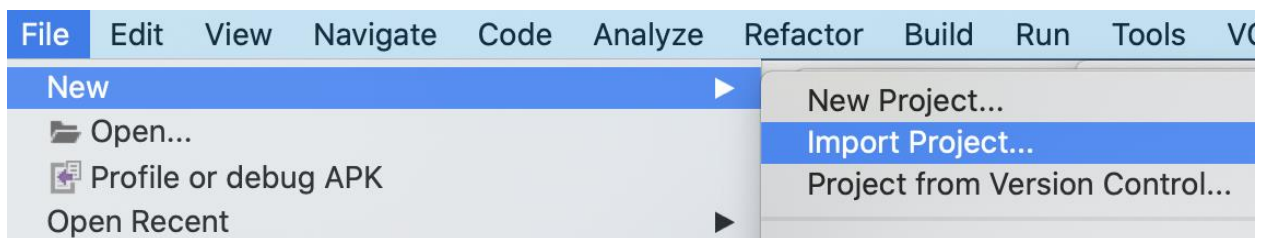
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. In the project folder, select the Happy Birthday folder.
5. Click Open.
6. Wait for Android Studio to open the project.



7. Click the **Run** button to build and run the app. Make sure it builds as expected.
8. Browse the project files in the **Project** tool window to see how the app is set-up.

6. Summary

- The **Resource Manager** in Android Studio helps you add and organize your images and other resources.
- An `ImageView` is a UI element for displaying images in your app.
- `ImageViews` should have a content description to help make your app more accessible.
- Text that is shown to the user like the birthday greeting should be extracted into a string resource to make it easier to translate your app into other languages.

7. Learn more

- [Vocabulary for Android Basics in Kotlin](#)
- [Resource Manager](#) in Android Studio
- [ImageView](#) class
- [Accessibility](#)
- [Support different languages](#)
- [Getting started with GitHub](#)

8. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab.

Do the following:

1. Create your own birthday card app based on your design.
2. Start by thinking about what `Views` you will need.
3. What order would be easiest to add them?
4. What images do you need to add to the drawable folder?

There are two types of bitmap images commonly used for Android apps, JPEG files and PNG files. PNG files can have transparent (empty) areas in them. Read more about image formats in [the developer reference](#).

5. Remember to position the `Views` first with constraints and margins, then style them.
6. To make the text stand out more on some images, try experimenting with the **`shadowColor`**, **`shadowDx`**, **`shadowDy`** and **`shadowRadius`**.



Check your work:

Your finished app should run without errors and show the birthday card you designed.

Congratulations, you finished creating your own Birthday Card app! Share your work on social media, and use the hashtag #LearningKotlin so we can see it!